Embodying AI Assistants in Live2D

Design Framework for Reactive Live2D Character Rigs Driven by Agentic LLM States

Version: 1.0
Author: John Abel - u3269020
Institution: University of Canberra
Project: Capstone - Emerging Production Technologies

> **Purpose.** This framework records the design decisions, parameter mappings, and transferable principles produced through the practice-led development of a Live2D character rig driven by real-time agentic LLM states. It is intended to serve as a reusable starting point, enabling a practitioner with Live2D experience to apply these principles to a different character model and a different agentic bot without starting from first principles.

Table of Contents

---

1. Introduction

1.1 The Problem

Modern agentic LLM systems are capable of multi-step reasoning and tool use, yet the interfaces that expose them to users remain static text boxes. When a bot is searching the

web, calling an API, reasoning through a problem, or returning an error, the user experience is identical in each case: a spinner, or the word "Working…".

This framework addresses a specific gap in the literature: **no published methodology exists for designing Live2D character rigs to communicate the internal states of an agentic LLM in real time, during the process rather than after output is produced.**

1.2 What This Framework Covers

- A vocabulary of six named display states derived from the agentic loop
- Per-state parameter mappings (static targets and drift ranges) developed and tested with the Live2D Niziiro Mao Example model
- A custom state JSON schema for encoding state behaviour independently of any specific model
- An animation system architecture for driving parameters at the SDK level
- Transferable design principles grounded in Embodied Conversational Agent (**ECA**) literature
- A log of what was tried and rejected, with root-cause analysis

1.3 What This Framework Does Not Cover

- LLM training, fine-tuning, or prompt engineering
- Voice or TTS integration
- Custom character artwork or rig construction (model-specific work)
- Production deployment or packaging

---

2. Literature Foundation

2.1 Embodied Conversational Agents (ECA)

The theoretical basis for this project draws from ECA research, particularly Pelachaud (2005), which establishes that effective non-verbal communication operates across multiple channels simultaneously: facial expression, body posture, gaze direction, timing, and rhythm, and that these channels must be synchronised to carry meaning convincingly.

Pelachaud distinguishes between two categories of non-verbal expression that are directly applicable to agentic state design:

Category	Definition	Application in This Framework
Communicative Act	"I am doing X" - signals current action or state to the observer	The primary mode for all six agentic states. Each state communicates what the bot is currently doing.

Category	Definition	Application in This Framework
Affective State	"I feel X" - signals emotion or attitude through expression	A secondary layer, expressions carry emotional colouring (concern in error, engagement in responding) but this is subordinate to the communicative intent.

Pelachaud also notes that **timing and rhythm independently affect the meaning of an expression**, a slow blink reads as contemplation; a fast one reads as alert attention. This finding directly informs the dual-sine drift system and the state-specific fade timings in this framework.

> *"We communicate through our choice of words, facial expressions, body postures, gaze, gestures etc. Nonverbal behaviors accompany the flow of speech and are synchronized at the verbal level, punctuating accented phonemic segments and pauses."* - Pelachaud (2005)

2.2 The ReAct Framework

The agentic loop architecture is grounded in Yao et al. (2023), which formalises the Reasoning + Acting (ReAct) pattern: the model alternates between reasoning traces ("I need to find X") and action steps ("search(X)"). This pattern maps directly onto the state vocabulary developed here: `thinking` corresponds to reasoning; `working` corresponds to acting / action execution.

2.3 The Gap

ECA research (Cassell, 2000; Pelachaud, 2005) establishes principles for embodied agents but operates exclusively in 3D and models conversational emotion rather than agentic process states. Tools such as Open-LLM-VTuber and Prompt-to-Animation (2025) exist at the intersection of LLMs and animated avatars, but both map to emotion tags embedded in **completed responses**, after the agent has finished reasoning and/or acting. No published framework addresses real-time state communication during the process, or provides a parameter-level design methodology for this use case.

---

3. Methodology

3.1 Practice-Led Research

This framework was produced through practice-led research: the artefact (the working prototype) is the primary research output, and the design decisions made during its construction constitute the research contribution. Design choices are documented with explicit rationale so they are transferable rather than merely descriptive.

3.2 Process Overview

1. **Research/Literature review** - ECA principles and existing LLM-avatar tools surveyed to identify the gap and establish theoretical grounding (ECA-1, ECA-2)
2. **State vocabulary definition** - candidate states identified by annotating the agentic loop source code at every significant transition point, mapped to communicative acts, and formalised into a six-state vocabulary
3. **Parameter authoring** - each state authored visually in Cubism Editor 5 by manipulating parameter sliders, then transcribed into a custom state JSON format
4. **Integration** - state controller implemented at the SDK level, driving parameters per-frame from state JSON definitions via a blend system
5. **Evaluation** - informal legibility testing with 13 peer observers; states assessed for readability without explanation (see Section 7)
6. **Iteration** - parameter values and timing adjusted based on observer feedback

---

4. State Vocabulary

4.1 Design Decisions

The vocabulary was derived directly from the agentic loop by annotating each significant transition point in the bot source code with a candidate state label. Technical states were then collapsed into display states through abstraction, mapping the number of technical states to the number of visual states where visual states < technical states, ensuring each displayed state is visually distinct and has sufficient dwell time to register with an observer.

> **Key Design Decision: Abstraction**
>
> The bot completes end-to-end in approximately 2 seconds. Without abstraction, states would flash for 200ms or less, becoming imperceptible to observers. Multiple technical states (`tool_selecting` , `tool_dispatching` , `awaiting_result`) are collapsed into a single `working` display state. A necessary design choice to maintain coherence during fast agentic loops.

4.2 State Summary

State	Entry Trigger	Min Display	Max Display	FadeIn	FadeOut	Mouth
`idle`	Bot at rest / ~3-5s after responding	-	-	800ms	800ms	No
`thinking`	Message received, API call in flight	500ms	-	400ms	400ms	No

State	Entry Trigger	Min Display	Max Display	FadeIn	FadeOut	Mouth
`working`	Tool dispatching, awaiting result	600ms	-	300ms	300ms	No
`responding`	`end_turn` stop reason / reply streaming	-	5000ms	400ms	500ms	Yes
`error`	Exception, rate limit, or API failure	600ms	4000ms	200ms	600ms	No
`surprise`	Notification received from server	500ms	2500ms	400ms	400ms	No

4.3 State Rationale

`idle`

The baseline state, everything else is read against it. Slow, organic drift with no directional signal. The slight positive `ParamMouthForm` (0.2) conveys resting contentment rather than neutral blankness. 800ms fade ensures idle always feels like a gentle return rather than a snap.

`thinking`

Gaze aversion (`ParamAngleX` -30, `ParamEyeBallX` -1, `ParamEyeBallY` 1) signals internal cognitive processing, drawing directly on Pelachaud (2005): agents using gaze aversion during complex reasoning are perceived as more natural and trustworthy than those maintaining direct eye contact. **Full eye closure was rejected**, it reads as sleep or shutdown rather than active processing. Reduced eye openness (0.7-0.9 drifting) preserves consciousness while reducing engagement intensity.

`working`

Forward body lean (`ParamAngleY` -30, `ParamBodyAngleY` -10) and rapid eye scanning (`ParamEyeBallX/Y` full drift, fast cycle) signal focused task execution. Arms raised to a working position (`ParamArmLA01/RA01` = 20). **Legibility finding: this state was misidentified by 62% of observers in isolation, but correctly identified in sequence.** See Section 7 for full analysis.

`responding`

Wide eyes (`ParamEyeLOpen/ROpen` 1.2), brows raised and drawn together, engaged forward posture. The mouth state machine activates (`UseMouth: true`), cycling through vowel phoneme shapes (`ParamA` , `I` , `U` , `E` , `O`) with randomised hold and rest durations to simulate speech without actual speech data. Maximum display time (5000ms) ensures the state persists if the bot is streaming a long response.

`error`

Fastest fade-in of all states (200ms), the abruptness of the transition itself carries meaning, signalling something unexpected. Shoulders raised, downcast posture, gaze averted downward, angry mouth parameters active. **Deliberately distinguished from** `thinking` by the downward rather than upward gaze direction. Maximum display 4000ms then returns to idle.

> **Error: Fade Timing Rationale**
>
> Error has the fastest fade-in (200ms) and the slowest fade-out (600ms). The fast fade-in signals abruptness. The slow fade-out ensures the distressed expression lingers rather than snapping cleanly to idle, conveying that the issue is not instantly resolved.

`surprise`

Added as a sixth state specifically for server-push notifications. Eye smile parameters, eye effect active, cheek flush, open-mouth O shape held. Short maximum display time (2500ms) ensures it snaps back to idle promptly. **100% observer identification rate**, the most legible state in testing.

4.4 Sequential vs Isolated Legibility

> **Key Finding**
>
> The `working` state was misidentified by 62% of observers when shown in isolation. The primary confusion was with `thinking`, observers noted the absence of mouth animation and interpreted the scanning behaviour as "planning" rather than "executing". When shown in sequence (thinking → working → thinking → responding), the state was correctly identified by all observers who later provided feedback.
>
> **This suggests that agentic state rigs should be evaluated in sequence, not in isolation.** States carry meaning relative to their neighbours, a state that is ambiguous alone may be unambiguous in context.

---

5. Parameter Mappings

All parameter values are based on the final state JSON files, after iteration from feedback and testing. Static parameters are held at fixed values for the duration of the state. Drift parameters oscillate continuously within their defined range using a dual-sine function (see Section 6.2). All static parameters use `Set`, acting as an `Overwrite` blend mode.

5.1 `idle`

FadeIn: 800ms | **FadeOut:** 800ms | **MinTime:** - | **MaxTime:** - | **Mouth:** No

Static Parameters

Parameter ID	Value	Rationale
`ParamEyeLOpen`	1.0	Full openness, resting alert baseline
`ParamEyeROpen`	1.0	Full openness
`ParamEyeLForm`	0.0	Neutral eye shape
`ParamEyeRForm`	0.0	Neutral eye shape
`ParamBrowLY`	0.0	Neutral brow height
`ParamBrowRY`	0.0	Neutral brow height
`ParamBrowLForm`	0.0	Neutral brow shape
`ParamBrowRForm`	0.0	Neutral brow shape
`ParamMouthForm`	0.2	Slight positive, relaxed contentment, not forced smile
`ParamAngleY`	0.0	Neutral head height, no directional signal
`ParamBodyAngleY`	0.0	Neutral body height
`ParamBodyAngleZ`	0.0	Neutral body roll

Drift Parameters

Parameter ID	Range	Cycle (s)	Rationale
`ParamAngleX`	-10 to 10	4	Slow lateral head drift, living presence
`ParamBodyAngleX`	-10 to 10	4	Body sway matching head
`ParamEyeBallX`	-1 to 1	3	Gentle horizontal eye gaze drift
`ParamEyeBallY`	-0.5 to 0.5	3	Gentle vertical eye gaze drift
`ParamLeftShoulderUp`	-10 to 10	3	Natural shoulder movement
`ParamRightShoulderUp`	-10 to 10	3	Natural shoulder movement
`ParamArmLA01`	5 to 5	5	Arm resting position hold
`ParamArmRA01`	5 to 5	5	Arm resting position hold
`ParamArmLA02`	0 to 10	2	Subtle arm movement
`ParamArmRA02`	0 to 10	2	Subtle arm movement
`ParamArmLA03`	-30 to 30	1	Fine arm articulation
`ParamArmRA03`	-30 to 30	1	Fine arm articulation

5.2 `thinking`

FadeIn: 400ms | **FadeOut:** 400ms | **MinTime:** 500ms | **MaxTime:** - | **Mouth:** No

Static Parameters

Parameter ID	Value	Rationale
`ParamAngleX`	-30	Head turned away, gaze aversion signals internal processing (Pelachaud, 2005)
`ParamAngleY`	30	Head tilted upward, "looking up while thinking"
`ParamEyeLForm`	1.0	Narrowed eye form, concentration
`ParamEyeRForm`	1.0	Narrowed eye form
`ParamEyeBallY`	1.0	Upward eye position, consistent with head tilt
`ParamBrowLForm`	0.5	Slight brow furrow, cognitive effort
`ParamBrowRForm`	1.0	Brow furrow, asymmetric for naturalness
`ParamMouthDown`	0.5	Slight downward mouth, thoughtful, non-committal

Drift Parameters

Parameter ID	Range	Cycle (s)	Rationale
`ParamAngleZ`	0 to 30	5	Random head tilt, unsettled, processing feel
`ParamBodyAngleX`	-5 to 5	5	Subtle body shift
`ParamBrowLY`	-1.0 to -0.5	7	Furrowed brow, sustained concentration
`ParamBrowRY`	-0.5 to 0	7	Asymmetric furrow
`ParamEyeLOpen`	0.7 to 0.9	6	Reduced but not closed, withdrawn without shutdown
`ParamEyeROpen`	0.7 to 0.9	5	Slight asymmetry
`ParamEyeBallX`	-1.0 to -0.75	12	Sustained leftward gaze aversion, slow cycle
`ParamMouthDown`	0.5 to 0.75	6	Subtle mouth movement
`ParamArmLA01`	-10 to 0	5	Arms lowered slightly during thought
`ParamArmRA01`	-10 to 0	5	Arms lowered slightly
`ParamArmLA02`	-5 to 5	2	Subtle arm articulation
`ParamArmRA02`	-5 to 5	2	Subtle arm articulation
`ParamArmLA03`	-30 to 30	1	Fine arm movement
`ParamArmRA03`	-30 to 30	1	Fine arm movement

5.3 `working`

FadeIn: 300ms | **FadeOut:** 300ms | **MinTime:** 600ms | **MaxTime:** - | **Mouth:** No

Static Parameters

Parameter ID	Value	Rationale
`ParamAngleY`	-30	Forward head lean, engaged with task
`ParamEyeLForm`	0.5	Focused eye shape, squinting with concentration
`ParamEyeRForm`	0.5	Focused eye shape
`ParamBrowLX`	-0.5	Brows drawn inward, focused frown
`ParamBrowRX`	-0.5	Brows drawn inward
`ParamBrowLAngle`	-0.2	Slight brow angle, effortful expression
`ParamBrowRAngle`	-0.2	Slight brow angle
`ParamBodyAngleY`	-10	Forward body lean, physically engaged
`ParamArmLA01`	20	Arms raised to working position
`ParamArmRA01`	20	Arms raised to working position
`ParamArmLA02`	-20	Arms extended forward
`ParamArmRA02`	-20	Arms extended forward

Drift Parameters

Parameter ID	Range	Cycle (s)	Rationale
`ParamAngleX`	-15 to 15	4	Lateral head scan, searching, reading
`ParamAngleZ`	-15 to 15	6	Head tilt while scanning
`ParamEyeBallX`	-1 to 1	8	Rapid horizontal eye movement, scanning
`ParamEyeBallY`	-1 to 1	7	Rapid vertical eye movement, scanning
`ParamI`	0.0 to 0.3	5	Subtle mouth, silent mouthing while working
`ParamMouthDown`	0.5 to 0.75	6	Concentrated mouth expression
`ParamBrowLY`	-0.6 to -0.5	3	Sustained furrowed brow
`ParamBrowRY`	-0.6 to -0.5	3	Sustained furrowed brow
`ParamEyeROpen`	0.75 to 0.85	3	Slightly narrowed, focused
`ParamEyeLOpen`	0.75 to 0.85	3	Slightly narrowed, focused

5.4 `responding`

FadeIn: 400ms | **FadeOut:** 500ms | **MinTime:** - | **MaxTime:** 5000ms | **Mouth:** Yes

Static Parameters

Parameter ID	Value	Rationale
`ParamEyeLOpen`	1.2	Wider than neutral, engaged, attentive to user
`ParamEyeROpen`	1.2	Wider than neutral
`ParamBrowLY`	-0.3	Slightly raised brows, warm, engaged
`ParamBrowRY`	-0.3	Slightly raised brows
`ParamBrowLX`	0.75	Brows spread, open, receptive
`ParamBrowRX`	0.75	Brows spread
`ParamBrowLForm`	1.0	Positive brow form, friendly
`ParamBrowRForm`	1.0	Positive brow form
`ParamBodyAngleY`	0.0	Return to neutral, task complete

Drift Parameters

Parameter ID	Range	Cycle (s)	Rationale
`ParamAngleX`	-3 to 3	6	Gentle lateral movement, engaged but relaxed
`ParamAngleY`	-15 to 15	5	Slight head movement while speaking
`ParamAngleZ`	-10 to 10	5	Head roll during speech
`ParamBodyAngleX`	-5 to 5	5	Body sway, natural speaking movement
`ParamBodyAngleZ`	-5 to 5	4	Body roll
`ParamMouthDown`	0 to 1.0	20	Very slow mouth drift, background expression
`ParamArmLA01`	5 to 5	5	Arms at neutral speaking position
`ParamArmRA01`	5 to 5	5	Arms at neutral speaking position
`ParamArmLA02`	0 to 10	2	Subtle arm movement while speaking
`ParamArmRA02`	0 to 10	2	Subtle arm movement
`ParamArmLA03`	-30 to 30	1	Fine arm articulation
`ParamArmRA03`	-30 to 30	1	Fine arm articulation

> **Mouth State Machine (UseMouth: true)**
>
> A phoneme state machine cycles through `ParamA`, `ParamI`, `ParamU`, `ParamE`, `ParamO`. Phases: `rest → open → hold → close → rest`. Hold duration: 60-140ms randomised. Rest duration: 40-160ms randomised. This produces speech-like mouth movement without actual speech data, avoiding the mechanical feel of a fixed animation cycle.

5.5 `error`

FadeIn: 200ms | **FadeOut:** 600ms | **MinTime:** 600ms | **MaxTime:** 4000ms | **Mouth:** No

Static Parameters

Parameter ID	Value	Rationale
`ParamAngleY`	-30	Head bowed, defeated posture
`ParamEyeBallX`	-1.0	Gaze averted downward-left, shame/concern
`ParamEyeBallY`	-1.0	Gaze downward
`ParamEyeBallForm`	-1.0	Soft eye form, distressed
`ParamBrowLX`	0.5	Inner brows raised, worried expression
`ParamBrowRX`	0.5	Inner brows raised
`ParamBrowLAngle`	-1.0	Brow angled inward-down, concern
`ParamBrowRAngle`	-1.0	Brow angled inward-down
`ParamBrowLForm`	-1.0	Distressed brow form
`ParamBrowRForm`	-1.0	Distressed brow form
`ParamMouthAngryLine`	1.0	Flat concerned line, distressed, not angry
`ParamBodyAngleY`	-10	Body withdrawn, recoil from error
`ParamBodyAngleZ`	-10	Body roll away
`ParamLeftShoulderUp`	10	Shoulders raised, defensive posture
`ParamRightShoulderUp`	10	Shoulders raised
`ParamArmLA01`	-10	Arms pulled in, retreating posture
`ParamArmRA01`	-10	Arms pulled in
`ParamArmLA02`	0	Arms relaxed inward
`ParamArmRA02`	0	Arms relaxed inward

Drift Parameters

Parameter ID	Range	Cycle (s)	Rationale
`ParamAngleX`	-30 to 0	6	Head averts left, avoidant, not confrontational
`ParamAngleZ`	-30 to 0	6	Head rolls away
`ParamEyeLOpen`	0.3 to 0.6	8	Eyes partially closed, distressed
`ParamEyeROpen`	0.3 to 0.6	8	Eyes partially closed
`ParamBrowLY`	-1.0 to 0.0	3	Brow rises and falls, ongoing worry
`ParamBrowRY`	-1.0 to 0.0	3	Brow rises and falls
`ParamMouthAngry`	0.75 to 1.0	12	Subtle mouth movement, distressed

5.6 `surprise`

FadeIn: 400ms | **FadeOut:** 400ms | **MinTime:** 500ms | **MaxTime:** 2500ms | **Mouth:** No

Static Parameters

Parameter ID	Value	Rationale
`ParamAngleX`	-30	Head turns away, startled recoil
`ParamEyeLSmile`	1	Eye smile, positive surprise, delight
`ParamEyeRSmile`	1	Eye smile
`ParamEyeBallX`	-1	Eyes dart to side, startled
`ParamEyeBallY`	1	Eyes dart upward
`ParamEyeEffect`	1	Eye sparkle, excited/delighted
`ParamBrowLX`	0.5	Brows raised inward, surprised
`ParamBrowRX`	0.5	Brows raised inward
`ParamBrowLForm`	1	Positive brow form
`ParamBrowRForm`	1	Positive brow form
`ParamBodyAngleX`	-10	Body leans away, physical recoil
`ParamLeftShoulderUp`	-10	Shoulder drops, asymmetric recoil
`ParamRightShoulderUp`	10	Shoulder raises, startled shrug

Drift Parameters

Parameter ID	Range	Cycle (s)	Rationale
`ParamAngleY`	-1.0 to 10	5	Head rises, looking up at notification
`ParamAngleZ`	-20 to -0.5	15	Head tilts, sustained surprised lean
`ParamCheek`	0.5 to 0.75	5	Cheek flush, pleased/embarrassed
`ParamEyeLOpen`	1.1 to 1.2	15	Wide eyes, sustained surprise
`ParamEyeROpen`	1.1 to 1.2	15	Wide eyes
`ParamEyeBallForm`	-1 to -0.9	10	Soft eye form, delighted
`ParamBrowLY`	0.25 to 0.5	10	Raised brows drift, settling surprise
`ParamBrowRY`	0.25 to 0.5	10	Raised brows drift
`ParamO`	0.75 to 1	20	Held open-mouth O shape, "oh!"
`ParamBodyAngleY`	5.0 to 10	20	Body leans back, recoil held
`ParamBodyAngleZ`	5.0 to 10	20	Body roll held
`ParamArmLA01`	5 to 15	10	Arms raise in surprise
`ParamArmLA02`	0 to 10	10	Arm articulation
`ParamArmRA01`	5 to 15	5	Arms raise
`ParamArmRA02`	0.125 to 10	10	Arm articulation

---

6. Animation System Architecture

6.1 State JSON Schema

All state behaviour is encoded in a custom JSON format, distinct from the standard `.exp3.json`. This format was developed because the standard format had no concept of drift ranges, timing gates, or mouth control.

Field	Type	Purpose
`FadeInTime`	ms (int)	Duration for blending into this state. Varies per state, error is 200ms (abrupt), idle is 800ms (gentle).
`FadeOutTime`	ms (int)	Duration for blending out of this state.
`UseMouth`	boolean	When true, activates the phoneme mouth state machine (ParamA/I/U/E/O cycling).
`MinimumTime`	ms (int)	Minimum display floor, state will not transition out until this time has elapsed.
`MaximumTime`	ms (int)	Maximum display ceiling, state auto-returns to idle after this duration. 0 = no limit.

Field	Type	Purpose
`Static`	array	Parameters held at fixed values. Each entry: `{ Id, Value }`. All use Overwrite blend.
`Drift`	array	Parameters animated continuously within a range. Each entry: `{ Id, Min, Max, Time }`. Time is the cycle period in seconds.

Example, thinking.json (abbreviated):

```
{
  "FadeInTime": 400,
  "FadeOutTime": 400,
  "UseMouth": false,
  "MinimumTime": 500,
  "MaximumTime": 0,
  "Static": [
    { "Id": "ParamAngleX", "Value": -30 },
    { "Id": "ParamEyeBallY", "Value": 1.0 }
  ],
  "Drift": [
    { "Id": "ParamAngleZ", "Min": 0, "Max": 30, "Time": 5 },
    { "Id": "ParamEyeLOpen", "Min": 0.7, "Max": 0.9, "Time": 6 }
  ]
}
```

6.2 Dual-Sine Drift

Rather than `Math.random()` per frame (jitter) or a single sine wave (mechanical repetition), drift parameters are driven by a dual-sine oscillator:

```
value = mid + (sin(t/t1 + p1) × 0.6 + sin(t/t2 + p2) × 0.4) × range
```

Two sine waves at different periods (`t1` , `t2`) and randomised phase offsets (`p1` , `p2`) are blended 60/40. Periods and phases are lazily initialised per parameter key with random values, ensuring all parameters drift independently and asynchronously. The result is non-repeating, organic-feeling motion.

6.3 State Timing Gates

Because the agentic loop can dispatch state changes faster than they are perceptible, a min/max display time system governs all transitions:

- `MinimumTime` : a state must be displayed for at least this long before a transition is allowed. Transitions that arrive early are queued as `_pendingState` and committed on the next eligible frame.

- `MaximumTime`: a state auto-returns to idle after this duration. Prevents getting stuck in `responding` or `error` indefinitely if the server drops a message.

6.4 Parameter Ownership & Blend System

On startup, the system snapshots the Live2D model's default value for every parameter referenced across all state JSON files. This baseline is used to resolve transitions:

- **Static params entering**: blend from live value to target static value over `FadeInTime`
- **Orphan params leaving**: parameters owned by the leaving state but not the arriving state blend back to their snapshotted default over `FadeOutTime`
- **Drift params shared between states**: the blend `to` value is updated every frame to match the current drift position, ensuring a seamless handoff when the blend completes rather than a stale snap

> **Critical Finding: `addParameterValueById` vs `setParameterValueById`**
>
> Early versions used `addParameterValueById`, which stacks values additively, causing parameters to accumulate beyond their intended ranges even when reset logic was operating correctly. Switching to `setParameterValueById` resolved this entirely. Any practitioner implementing a similar system should use set (overwrite) rather than add for all driven parameters.

6.5 SDK Architecture

The Live2D Web SDK (v5) is structured as a prebuilt application framework, not a modular library. The approach taken was surgical removal from the official sample application (CubismWebSamples):

- **Removed:** MotionManager, ExpressionManager, TouchManager, multi-model/multi-canvas logic, background rendering
- **Retained:** Cubism renderer and model pipeline, delegate lifecycle, update loop

The `AgentStateControl` class was owned at the Subdelegate level and accessed via reference from `LAppModel`, consistent with how other systems are accessed in the SDK. A minimal global API was exposed for external control:

```
(window as any).Live2DApp = {
  setState: (s: string) =>
    app.getSubdelegate().getAgentStateControl().setAgentState(s)
};
```

---

7. Legibility Testing

7.1 Methodology

Informal observer legibility testing was conducted to assess whether state expressions were readable without explanation to cold observers. The tool presented each state in randomised order and asked observers to select which state label best described what they saw, with an optional freetext reason field.

- **Sessions:** 13
- **Total responses:** 74
- **Testing dates:** 24-28 April 2026

7.2 Results by State

State	Correct	Total	Accuracy	Misidentified As
`surprise`	10	10	**100%**	-
`responding`	12	13	**92%**	idle ×1
`idle`	11	13	**85%**	responding ×2
`thinking`	11	13	**85%**	idle ×1, error ×1
`error`	9	12	**75%**	thinking ×2, responding ×1
`working`	5	13	**38%**	thinking ×5, idle ×1, responding ×2
Overall	**58**	**74**	**78.4%**	

7.3 Notable Observer Comments

idle

> *"It is Idling / not doing much, just looking back and forwards."*
> *"its a classic idle animation where a character looks attentive but not thinking about anything in particular"*
> *(Misread as responding) "Eye contact, open stance, gives 'interacting with you' vibes... movement comes across as excited rather than attempting to avoid the interaction"*

thinking

> *"Confused and debating something"*
> *"Overlap between thinking/planning and giving a response. Appears to be thinking of how to respond to an awkward question. Slightly narrowed eyes and thinned lips leans to thinking."*
> *"the head tilt and eyes squint is a sure sign of someone thinking or planning something"*

working (misidentifications)

> *"I think it is thinking / planning, but looks kinda disappointed to me"*
> *"this could also be an idle pose as there is no mouth animation or deep expression"*
> *"minimal body movement and forward facing leads to some level of directness. Lack of hand involvement pushed from focusing on a task to giving a response"*

responding

> *"the alertness on the face and the moving mouth animation truly makes it looks like the character is giving a response"*
> *"Moving mouth = speaking. Eye contact, forward facing torso, and open stance."*
> *"She talking to me in a regular anime manner."*

error

> *"She looks upset, like something went wrong or I said something wrong (Sorry Virtua-chan!!)"*
> *"Downcast eyes, pulling up bottom lip (pouting), reads something hasn't gone their way."*
> *(Misread as thinking) "Head down and slightly contemplative"*

surprise

> *"Wider eyes, slack jaws, transition from initial stance to new one was sudden."*
> *":o face and big eyes."*
> *"Wide eyes, mouth open, leaning back as if in shock"*

7.4 Key Findings

Finding 1: Working state isolation problem (38% accuracy)

The `working` state was the only state below 50% accuracy in isolation. All eight misidentifications fell into either `thinking` (×5) or `idle` / `responding` (×3). Observer comments consistently pointed to two causes:

1. **No mouth animation**: observers using mouth movement as the primary cue for active engagement (vs idle) found the absence disqualifying
2. **Scanning behaviour ambiguous**: rapid eye and head movement was interpreted as either thinking (planning) or idle (looking around) rather than task execution

This result drove a parameter adjustment increasing the arm pose contrast (arms raised and extended for working) to add a clearer body language signal distinct from thinking.

Finding 2: Responding state mouth as primary cue (92% accuracy)

Nearly all correct `responding` identifications explicitly cited mouth movement as the deciding cue. This validates the mouth state machine as the most effective legibility mechanism in the system and supports `UseMouth: true` as the primary differentiator for the responding state.

Finding 3: Idle misread as responding (×2)

Two observers identified idle as responding, citing "open stance" and "eye contact" as cues for interaction. This suggests the idle expression is at the edge of the boundary between resting attentiveness and active engagement, a considered design decision (the slight positive MouthForm and open eyes are intentional) but one that creates some ambiguity.

Finding 4: Sequential context improves working accuracy

Observers who saw the states in a sequence that included thinking → working reported the working state as immediately distinguishable. The arm position and forward lean that are ambiguous in isolation are unambiguous following the head-tilt-and-gaze-aversion of thinking. This finding supports evaluating agentic state rigs in sequence rather than isolation.

---

8. What Was Tried and Rejected

8.1 Expression Design

What Was Tried	Result	Rejection Reason / Outcome
Full eye closure for `thinking`	Rejected after informal test	Reads as sleep or shutdown. Reduced openness (0.7-0.9) preserves consciousness.
Identical expressions for `thinking` and `working`	Not distinguishable by observers	Key differentiator: gaze direction (up-away vs forward-scan) and body lean direction.
Motions for body animation	Conflicts with code-driven parameters	Cubism motion system writes to the same parameter buffer as code. All body animation moved to code-driven drift.
Standard `.exp3.json` expression files	Replaced with custom state JSON	Standard format has no concept of drift ranges, timing gates, or mouth control.
Ren'py with Live2D integration	Not suitable	Talking to external sources (sockets/layers) is not straightforward; would require middle-man workarounds.
`live2d-py` unofficial Python SDK	Rejected, lack of support	Non-official port, no tutorials, limited documentation, not maintained.

8.2 SDK Integration

What Was Tried	Result	Rejection Reason / Outcome
`live2dcubismframework.js` via plain `<script>` tag	`SyntaxError: Cannot use import statement outside a module`	Framework uses ES modules, requires bundling through webpack to use as include.
Webpack bundle (custom)	Bundle compiled; white-square rendering at runtime	Shader async loader uses hardcoded relative path (`../../Framework/Shaders/WebGL/`) that resolves incorrectly outside the demo app directory structure.
Polling `_isShaderLoaded` externally (webpack)	Timeout, shader instance never populated	`getShader(gl)` returns null before `setGlContext()` is called. Cannot poll the correct instance without access to internal SDK state.
Vite bundle (custom)	Same shader path failure	`setShaderPath()` called after `loadShaders()` is already called inside `startUp()`, path override arrives too late.
Node 18 for Vite build	Build failure	Vite 8 requires Node 20+. Resolved by installing Node 20 via nvm.
CubismWebSamples demo app (iframe, unmodified)	Worked, shaders load correctly; wrong model, wrong UI	`copy_resources.js` copies shaders to paths matching the hardcoded default. Demo loads 10 models, has cog-wheel UI and background image.
Extracting SDK as modular library	Repeated renderer initialisation failures	SDK has tightly coupled internal dependencies. Surgical removal is more reliable than extraction.
Final approach: strip the demo, inject custom logic	**Working**	Use official sample app as base. Remove MotionManager, ExpressionManager, TouchManager, multi-model logic. Inject `AgentStateControl` into the update loop. Expose `setState()` globally.

---

9. Transferable Design Principles

The following principles are derived from the specific decisions made during this project and are intended to generalise beyond the Niziiro Mao model and the Virtua bot.

P1; Display-State Abstraction is Mandatory for Fast Agents

Map N technical states to M visual states where M < N. For a bot completing in ~2 seconds, a minimum floor of 400-600ms per state is necessary to make states perceptible. Without abstraction, states flash invisibly.

Supported by: fast agent problem (observed), Pelachaud (2005) on timing.

P2; Gaze Direction Encodes Cognitive State

Upward/lateral gaze aversion signals internal processing. Downward gaze signals distress. Forward gaze with wide eyes signals engagement. These distinctions must be maintained across character designs to preserve legibility.

Supported by: Pelachaud (2005), legibility test results (thinking 85%, error 75%).

P3; Transition Speed Carries Meaning

Fast fade-in signals abruptness (error: 200ms). Slow fade-in signals deliberateness (idle: 800ms). The transition speed is a core part of communicating the states, not only used for visual smoothing.

Supported by: error/idle fade timing contrast.

P4; Sequential Legibility Differs from Isolated Legibility

States should be evaluated in the context of a realistic sequence, not as isolated snapshots. A state that is ambiguous alone may be unambiguous in context. Design for the sequence, not the frame.

Supported by: working state finding (38% isolated → high sequential).

P5; Unify Animation Control in a Single System

Rather than splitting face animation (expression files) from body animation (code), a single custom state JSON format and unified parameter driver handles both. This eliminates the class of conflicts that arise when two systems write to the same parameter buffer - the SDK's expression manager and a custom code-driven system fighting over the same values each frame.

The standard Cubism expression and motion pipeline was removed entirely in favour of direct per-frame parameter writes via setParameterValueById() driven from the state JSON definitions.

Supported by: expression manager conflict (attempted, rejected), custom state JSON architecture, parameter ownership blend system.

P6; Mouth Movement is the Strongest Legibility Cue

Observer data shows that the presence or absence of mouth animation was the deciding cue for a large proportion of correct identifications. The responding state (UseMouth: true) achieved 92% accuracy; the working state (no mouth) achieved 38%. Where the communication intent is "currently speaking/responding", mouth animation should be considered mandatory.

Supported by: legibility test results, observer comments.

P7; Organic Drift Over Repetition

Single sine wave oscillators produce mechanical, obviously artificial motion. Dual-sine oscillators with randomised phase and period offsets produce non-repeating motion that reads as natural presence. The 60/40 blend ratio and coprime period selection are the key parameters.

Supported by: drift system implementation and visual comparison.

P8; SDK Architecture: Strip, Do Not Extract

The Live2D Web SDK is a prebuilt application framework with tightly coupled internal dependencies. Attempting to extract it as a modular library leads to repeated rendering failures. Use the official sample app as a base and surgically remove unwanted systems rather than rebuilding from scratch.

Supported by: four failed custom integration attempts.

P9; Parameter Ownership Must Be Explicit at Transition

When a state transition occurs, parameters owned by the leaving state but not the arriving state must be explicitly returned to their defaults. If left unmanaged, orphan parameters accumulate at previous state values across multiple transitions. Always use `setParameterValueById` (overwrite), never `addParameterValueById` (accumulate).

Supported by: blend system design and addParameterValueById bug discovery.

---

10. Applying This Framework

Step 1; Define the agentic loop state vocabulary

1. Annotate your bot's source code at every significant transition point with a candidate state label
2. Collapse technical states into display states (target 5-7 states)
3. Define entry trigger, exit trigger, minimum display floor, and maximum display ceiling for each state

Step 2; Author parameter mappings for your character

1. Open your model in Cubism Editor 5 and explore the available parameters
2. For each state, manually move sliders to establish the target expression
3. Categorise each parameter as Static (fixed value) or Drift (range + cycle time)
4. Record parameter IDs and values using the state JSON schema in Section 6.1
5. **Note:** parameter IDs vary significantly between models. The mappings in Section 5 are for Niziiro Mao and should be treated as structural templates, not literal values.

Step 3; Implement the animation system

1. Use the CubismWebSamples demo app as a base, do not attempt to build a custom renderer
2. Strip to the minimum: single model, single canvas, no motion system, no expression manager, no touch manager
3. Implement `AgentStateControl` with blend system, drift system, and mouth state machine
4. Load state JSON files from a manifest at runtime to allow tuning without rebuilding
5. Expose a single `setState(stateName)` function on the global `window` object

Step 4; Connect the bot

1. Emit named state events from the bot at each transition point
2. Use `asyncio.sleep(0)` after each state emission to flush the WebSocket frame before synchronous work begins
3. Wire the WebSocket message handler to call `setState()` in the frontend

Step 5; Evaluate legibility

1. Show the prototype to cold observers without explaining the states
2. Ask them to describe what the character is doing in each state
3. Evaluate in sequence, not just in isolation
4. Iterate on any state that is consistently misidentified
5. Pay particular attention to states that lack mouth animation, these are most likely to be ambiguous

---

11. References

Cassell, J. (2000). *Embodied conversational agents.* MIT Press.

Live2D Inc. (n.d.). *Cubism SDK for Web.* https://www.live2d.com/en/sdk/download/cubism/

Live2D Inc. (n.d.). *CubismWebSamples.* https://github.com/Live2D/CubismWebSamples

Pelachaud, C. (2005). Multimodal expressive embodied conversational agents. *Proceedings of the 13th ACM International Conference on Multimedia,* 683-689. https://doi.org/10.1145/1101149.1101301

Yao, S., Zhao, J., Yu, D., Du, N., Shafran, I., Narasimhan, K., & Cao, Y. (2023). ReAct: Synergizing Reasoning and Acting in Language Models. *arXiv preprint arXiv:2210.03629.* https://arxiv.org/abs/2210.03629

Appendix A - State JSON Files

All six state JSON files and the full Live2D integration source are available at: https://github.com/devnanix/live2d-embodied-avatar

The repository includes setup instructions for running the integration locally. SDK files (Core/, Framework/) must be downloaded separately from https://www.live2d.com/en/sdk/download/cubism/ due to license restrictions.

Appendix B - Legibility Testing

Legibility testing was conducted via a custom tool built into the Virtua web interface. The tool presents each agentic state in randomised order and records the observer's identification choice, an optional freetext reason, and a correctness flag against the ground truth.

The tool remains live and open for independent observation at: https://virtua.nanix.dev/legibility

Full results and analysis from the 13-session, 74-response study are documented in Section 7 of the design framework.

Appendix C - Expression Attribution

Character model: Niziiro Mao - Live2D Inc. Used under the Live2D Free Material Licence Agreement for non-commercial educational purposes. https://www.live2d.com/en/terms/live2d-free-material-license-agreement/